TravelPlanner 백엔드 알고리즘 실행 순서

작성일: 2026-10-09 · 대상: 현재 PC에서 실행하는 TravelPlanner_Backend

이 문서는 C:₩Users₩USER₩Desktop₩TravelPlanner_GitHub_refresh₩TravelPlanner_Backend의 실제 호출 코드를 기준으로 작성했습니다. GitHub 최신 원본과 로컬 수정본은 서로 다릅니다. 마지막 부분에 원격 확인 결과를 따로 정리했습니다. 비밀번호와 개인 키는 포함하지 않습니다.

1. 전체 흐름

```
사용자 여행 조건 입력
  → 인증 및 백그라운드 작업 등록
  → 입력 조건 검증
  → DB 장소 후보 조회 / 부족하면 추가 검색
  → 도시별 장소 적합도 평가
  → 입출국 지점 확정 / 도시 방문 순서와 일차 배정
  → 날짜별 숙소 확정
  → 하루 일정 시뮬레이션
      이동 → 영업시간 → 식사 간격 → 고정 일정 → 복귀 가능 여부
  → 부족한 일차 후보 보강 / 재계산
  → 숙소·입출국·고정 일정 항목 조립
  → 설명·교통비 추정·확보된 경로 좌표 보강
  → 일정·방문 항목·교통 정보 DB 저장
  → 지상 교통비 / 항공비 분리 / 패스 비교
  → 완료 상태 반환 / 프론트가 결과 조회와 완료 팝업 표시
```

현재 핵심은 도시 방문 순서를 정하는 동적 계획법과 하루 방문 후보를 하나씩 고르는 제약 기반 탐욕 알고리즘입니다. 생성의 중심에서 Gemini가 전체 일정을 써 주는 방식은 사용하지 않습니다. 장소 검색·교통 공급자 등 외부 서비스는 설정과 데이터 확보 상태에 따라 호출될 수 있습니다.

2. 실제 처리 순서

01. 생성 요청을 백그라운드 작업으로 등록

PlanGenerationController.submit → PlanGenerationJobs.submit → PlanController.createPlan

프론트는 여행 조건과 clientRequestId를 보냅니다. 서버는 로그인 계정과 요청 ID를 묶어 작업을 관리합니다. 동일 ID와 동일 조건의 재요청은 기존 작업을 반환하고, 같은 ID로 조건이 바뀌면 충돌 처리합니다. 저장된 요청도 계정별로 찾아 중복 생성을 줄입니다.

- 한 사용자에게 미완료 작업은 최대 2개입니다.
- 서버 실행 스레드도 2개이며, 대기열은 8개입니다. 사용자별 제한과 서버 전체 처리량 제한은 다릅니다.
- 작업 상태는 QUEUED → RUNNING → SUCCEEDED / FAILED / CANCELLED입니다.
- 실행 대상으로 배정된 작업은 300초 후 중단을 시도합니다. 이미 저장 확정 단계에 들어갔다면 중단하지 않습니다.
- 진행 상태는 메모리에 있습니다. 재시작하면 실행 중인 작업 자체는 이어지지 않습니다. 저장 완료된 요청은 DB에서 다시 조회할 수 있습니다.

02. 입력 조건을 검증하고 기본값을 결정

PlanController.createPlan, PlanValidationService

여행 시작일·종료일, 방문 도시, 고정 일정 날짜·시간, 숙박 기간, 선호/제외 테마 충돌, 하루 활동 시간, 일정 제목 길이를 검사합니다. 로그인 계정에 해당하는 회원도 확인합니다.

숙소 미지정 날짜는 추천 대상이 되도록 suggestHotel=true로 설정합니다. 섬 간 이동수단이 없으면 비행기를 기본값으로 하고 비행기·배만 허용합니다. 공항을 입력하고 입출국 시각을 비워 두면 하루 시작·종료 시각을 사용합니다. 하루 시간이 없으면 09:00 / 22:00입니다.

03. 사용 가능한 여행 시간을 계산

PlanService.dayWindow, calculateTotalPhysicalMinutes

중간 일차는 하루 시작부터 종료까지를 사용합니다. 첫날 입국 시간이 있으면 입국 시간에 120분을 더해 활동 시작을 잡습니다. 마지막 날 출국 시간이 있으면 하루 종료와 출국 180분 전 중 더 이른 시간을 활동 마감으로 잡습니다. 1일 일정이면 두 조건을 함께 적용합니다.

따라서 일반 날의 시작·종료는 숙소 출발·도착 목표이고, 첫날·마지막 날은 공항 조건으로 활동 범위가 달라집니다. 생성에서는 이 범위 안에 숙소나 출국 공항까지의 이동도 포함할 수 있는지 검사합니다.

04. DB에서 방문 후보를 모으고 부족하면 보강

PlanController.createPlan, recoverPlaces

선택 도시의 place를 조회합니다. 주소를 이용한 도시 분류를 정규화하고, 선택 도시 밖 장소·시스템 항목·공항·숙소·좌표가 없는 장소를 초기 방문 후보에서 제외합니다.

목표 후보 분량은 대략 다음과 같습니다.

```
필요 분량 = max(0, 전체 활동 가능 시간 - 고정 일정 시간)
후보 목표 분량 = 필요 분량 × 1.5
현재 후보 분량 = 각 후보의 예상 체류시간 합계
```

목표에 부족하면 선택한 도시를 번갈아 검색합니다. 선호 테마가 없으면 필수 관광지·인기 맛집·랜드마크를 검색어로 사용합니다. 도시 수에 비해 기간이 길고 근교 제외가 꺼져 있으면 근교 검색어도 사용합니다. 초기 보강 시도 한도는 max(3, 선택 테마 개수)입니다.

새 장소는 실제 Place ID와 좌표를 확인하고 도시 분류를 교정한 뒤 중복 없이 저장합니다. 검색 실패가 곧바로 생성 전체 실패를 뜻하지는 않습니다. 기존 후보로 이어서 시도합니다.

05. 도시별 후보 적합도를 평가

PlanService.applyWeightedScoring

도시마다 따로 점수를 계산해 한 도시의 중심이 다른 도시 후보를 과도하게 감점하지 않도록 합니다. 제외 테마가 포함된 장소는 먼저 제거합니다.

항목	현재 처리
기본 점수	50
선택 테마 일치	일치하는 테마마다 +60

항목	현재 처리
가족 동행	관광지 +40, 액티비티 테마 -50
도시 기준점에서 10km 초과	초과 거리 1km당 대략 -5
최종 점수	양수인 후보만 남기고 높은 점수 순으로 정렬

기준점은 도시 후보 좌표의 중앙값이고, 중심에서 20km 이내의 역 후보가 있으면 그 역 좌표를 사용하기도 합니다. 이 점수는 후보 제외·정렬에 사용됩니다. 이후 실제 방문 선택은 별도의 이동·대기 비용으로 결정하므로, 테마 점수가 가장 높은 장소가 항상 첫 방문지는 아닙니다.

기본 점수 함수에 문자열 날씨 가감 코드가 있지만 생성 컨트롤러는 현재 날씨를 대신 넣지 않습니다. 실제 방문 시간대 예보 가중치는 뒤의 방문 비용 계산에서 적용합니다.

06. 공항과 도시 중심을 결정

PlanController.resolveAirport, airportCityAnchor, CityDayPlanner.centers

입국·출국 공항을 각각 확인하고 도시별 후보 좌표 중앙값으로 대표 위치를 만듭니다. 공항을 확보하면 공항 좌표를, 공항 대신 도시를 입력한 경우에는 도시 기준점을 방문 순서의 출발·도착 기준으로 사용합니다.

도시 기준점은 실제 교통 허브나 도로망을 최적화한 위치가 아니라 후보 좌표의 대표값입니다.

07. 도시 방문 순서와 일차를 배정

CityDayPlanner.citiesForDays

중복 도시를 제거합니다. 선택한 도시 수가 여행 일수보다 많거나 위치 자료가 부족하면 실패시키며, 한 번에 최대 15개 도시를 처리합니다.

1. 입국 기준점에서 가장 가까운 선택 도시를 첫 도시로 고정합니다. 입국 기준점이 없으면 첫 선택 도시를 기준으로 시작합니다.

2. 방문한 도시 집합과 마지막 도시를 상태로 하는 동적 계획법으로 도시 사이 거리 합이 작은 순서를 찾습니다.

3. 마지막 도시에서 출국 기준점까지의 거리도 종점 선택에 포함합니다.

4. 결정한 순서를 연속된 일차에 배정합니다. 현재는 dayIndex × 도시수 ÷ 전체일수 방식으로 대략 균등 배분합니다.

여기서 최소화하는 값은 도시 대표점 사이의 직선거리입니다. 실제 철도·버스·항공 시간과 운임을 넣어 도시 순서를 전역 최적화하는 단계는 아닙니다. 도시별 볼거리 분량에 따른 일수 재배분도 현재 호출 경로에는 없습니다.

08. 날짜별 숙소와 출발·도착 지점을 결정

StayPlanner.resolve

그 날짜에 사용자가 지정한 숙소가 있으면 이름과 기간을 기준으로 DB 또는 추가 검색에서 확인합니다. 미지정이면 해당 도시의 기존 숙소 최대 10개 중 후보 목록의 마지막 장소에 가까운 숙소를 먼저 고르고, 확보하지 못하면 숙소를 검색합니다. 이는 실제 하루 최종 방문지로 확정한 뒤 숙소를 고르는 방식과는 다릅니다.

숙소 선택은 도시·이름 기준 캐시로 중복 검색을 줄입니다. 좌표를 확보하지 못하면 숙소 미정 또는 위치 미확인 안내를 남기고, 왕복 이동시간을 확정하지 않습니다. 마지막 날에는 새 숙박을 추천하지 않습니다.

출국 전날에서 마지막 날 도시로 이동이 180분 이상이고 수동 숙박이 없으면, 전날 숙소를 다음 도시 쪽으로 잡아 긴 이동을 전날 저녁에 배치하려고 시도합니다.

09. 각 날에 넣을 후보를 준비

PlanController.createPlan

해당 일차 도시의 후보 중 여행에서 이미 사용한 Place ID를 제외합니다. 식당 후보는 관광지 점수가 높아도 함께 남겨 둡니다. 첫날 출발 기준은 입국 공항이며, 이후 날은 전날 숙소를 사용합니다. 마지막 날 도착 기준은 출국 공항, 나머지 날은 그날 숙소입니다.

10. 다음 방문 후보마다 이동 정보를 계산

TransportService.between, 각 TransitRouteProvider

같은 생성 작업 안에서는 장소·좌표·이동 수단·출발 시각을 묶어 이동 계산을 캐시합니다. 서로 다른 섬으로 판단되면 선택한 비행기/배의 이동 추정을 먼저 처리합니다. 국내선 운임 자료가 해당 구간에 있으면 적용합니다.

대중교통에서는 사용 가능한 공급자를 순서대로 조회합니다. 실시간 공급자 결과가 확보되면 우선 반환하고, 수집한 철도·버스·GTFS 결과끼리는 소요시간이 짧은 대안을 고릅니다. 조회 실패는 다른 공급자와 추정 경로로 이어집니다.

자료가 없으면 공항 이동·장거리 도시 이동·일반 대중교통·자동차·도보를 구분해 거리와 속도로 추정합니다. 직선거리 보정 1.25가 사용됩니다. 도보 표시 소요시간은 최소 3분입니다. 미확인 대중교통 운임은 거리 구간별 계획용 평균으로 보완하며, 사업자·노선을 새로 지어내지는 않습니다.

즉 '요금과 시간이 있음', '사업자와 노선이 확인됨', '실제 경로 좌표가 있음'은 각각 다른 상태입니다.

11. 하루 일정 시뮬레이션으로 다음 장소를 선택

PlanService.runScheduleSimulation

현재 시각에서 아직 방문하지 않은 후보를 반복해서 확인합니다.

1. 식사·간식·쇼핑 횟수 제한을 검사합니다.

2. 현재 장소에서 후보까지 이동한 도착 시각을 계산합니다.

3. 방문 날짜의 영업시간과 도착 시각 중 늦은 시각을 방문 시작으로 잡습니다.

4. 식사·간식·야경 시간 조건을 적용합니다.

5. 고정 일정 앞뒤 30분과 겹치면 해당 고정 일정 종료 이후로 시작을 이동합니다.

6. 체류시간과 다음 활동용 완충 시간을 더해 방문 가능 여부를 검사합니다.

7. 그 장소를 방문한 뒤 숙소 또는 출국 공항으로 마감 안에 돌아갈 수 있는지도 검사합니다.

8. 가능한 후보 중 식사 시점의 식당을 우선하고, 이동·대기·되돌아감 비용이 낮은 후보를 선택합니다.

현재 방문 선택 비용은 다음과 같습니다. 낮을수록 우선합니다.

```
비용 = (방문 시작 - 현재 시각)
      + 이동시간 × 2
      + 되돌아감 거리 패널티
      - 방문 시간대 날씨 점수

되돌아감 패널티 = max(0, 이전이전→이전 + 이전→후보 - 이전이전→후보 거리) × 3
```

후보를 넣은 뒤 현재 시각·이전 장소·횟수를 갱신하고 반복합니다. 다음 식사 가능 시간을 긴 관광 방문이 막아 버리면 그 전에 끝낼 수 있는 다른 활동을 찾거나 식당을 먼저 배치합니다. 이동하며 이미 떠난 도시로 다시 돌아가는 후보는 제거합니다.

12. 체류시간·식사·날씨 규칙을 적용

체류시간: DB의 추천 체류시간이 있으면 우선 사용합니다. 없으면 기본 90분, 쇼핑 120분, 테마파크 480분, 식음 60분 등을 사용합니다. 가족/힐링은 체류시간을 1.2배 하고 일반 식사는 최소 45분입니다.

완충 시간: 가족·부모님·힐링 30분, 쇼핑·액티비티 10분, 일반 20분입니다. 하루 일정 강도 필드가 있다고 해서 모든 계산에 별도 배율로 적용되는 것은 아닙니다.

식사: 기본 2회이며 일반 테마는 1~3회, 맛집/미식여행은 최대 5회로 제한합니다. 식사 시작 간격은 기본 300분, 최종 식사 횟수가 5인 경우 150분입니다. 간식은 별도 최대 2회, 간식 시작 간격은 180분이고 식음 방문 종료 후 다음 식음 시작은 최소 60분입니다. 지정 횟수는 목표/상한이며 데이터와 영업시간·일정 시간 부족 시 반드시 전부 채우는 보장은 없습니다.

쇼핑: 기본 1회, 쇼핑 테마는 최대 3회입니다. 이름이나 테마에서 야간 장소로 판단한 후보는 17:30 이후 시작하도록 제한합니다.

날씨: 일본 시간의 실제 방문 시각에 대응하는 3시간 예보가 있고, 캐시 수집 후 36시간 이내인 경우만 적용합니다. 강수확률과 악천후 코드로 최대 약 60점의 가중치를 계산해 실내는 유리하게, 실외는 불리하게 합니다. 해당 시간의 예보가 없으면 0점이며 오늘 날씨를 미래 여행 날짜에 대신 쓰지 않습니다.

13. 부족한 일차를 보강하고 다시 계산

PlanController.needsRecovery, recoverPlaces

식사나 일정 충실도가 부족한 날은 추가 후보 확보와 하루 시뮬레이션을 최대 2회 더 시도합니다. 기존안보다 식사 횟수가 늘거나, 같은 식사 횟수에서 활동 충실도가 높아지거나 대기가 줄어든 안을 채택합니다.

전체 생성 재시도 상한은 3회입니다. 방문 가능한 활동이 없고 고정 일정을 제외한 시간도 180분 이상 남는 날은 422 오류로 중단합니다. 하루 전체를 여유 시간으로만 구성해 정상 일정으로 저장하지 않습니다.

14. 숙소·입출국·고정 일정 항목을 조립

DayBoundaryService.append, DailyEventPlanner, TimelineOrder

숙소 출발·도착, 입국 심사와 수하물 수령, 출국 공항 이동을 방문 항목과 함께 만듭니다. 긴 도시 간 이동은 별도 시간 블록으로 표현합니다. 사용자의 고정 일정 날짜·시각도 별도 기록으로 추가합니다.

90분 이상 영업 시작을 기다려야 하고 고정 일정과 겹치지 않는 경우는 대기·휴식 항목이 생길 수 있습니다. 후보와 추가 검색으로 해당 시간에 맞는 활동을 확보하지 못했을 때의 대체 항목입니다. 결과는 일차·시각·역할 순서로 정렬하고 공항 좌표를 연결합니다.

15. 응답 정보를 보강하고 DB에 저장

KoreanDescriptionService, DistanceTransportEstimateService, RouteGeometryService, PlanController

설명 번역, 운임 누락 보완, 이미 확보한 경로 좌표 연결을 수행합니다. 경로 좌표는 도보망·철도망·공식 GTFS shape 등 자료가 있을 때만 연결합니다. 자료가 없는 구간에 직선 경로를 만들어 실제 노선처럼 저장하지 않습니다. 현재 프론트 지도는 핀 사이 파란 선을 표시하지 않고, 핀에서 다음 장소까지 Google 지도 경로를 열도록 구성돼 있습니다.

그다음 제목·기간·생성 조건 스냅샷과 방문 순서를 저장합니다. 빈 제목은 첫 도시 ~ 마지막 도시 여행으로 구성합니다. 일반 방문은 Place ID로 기존 장소와 연결하고, 공항·숙소·고정 일정 등은 역할과 위치 정보를 가진 항목으로 처리합니다. 저장 과정에서 주소·전화·영업시간 등 장소 상세 정보를 채웁니다.

저장 구조는 plan → itinerary → 각 항목의 traffic입니다. 교통 종류·소요시간·비용·경로와 표시 메타데이터도 함께 보관합니다. 이 과정은 트랜잭션이며 저장 확정 직전에 취소 여부를 다시 확인합니다.

16. 비용과 패스를 비교하고 완료 상태를 반환

TransitCostService.summarize, PlanGenerationJobs

지상 교통비와 항공비를 분리합니다. 국제선 비용을 국내 대중교통비에 합치지 않습니다. 운임이 없으면 무료로 취급하지 않고 미확인 구간으로 남깁니다.

패스는 유효한 수집 자료와 실제 확인된 사업자·노선·도시 범위를 이용해 비교합니다. 지역권은 해당 노선/기간을, 전국·지역 JR권은 연속 사용 일수를 고려합니다. 같은 이동비를 여러 패스로 중복 할인하지 않도록 구간 적용 관계를 비교하고 절약액이 큰 조합을 선택합니다. 일반 승차권보다 비싼 패스는 추천하지 않습니다.

사업자·노선이 미확인인 거리 평균 구간을 JR 이동으로 가정해 전국권을 추천하지 않습니다. 이런 구간이 있으면 패스 비교 미완료로 표시합니다. DB에 상품을 적재했다는 사실만으로 자동 추천 가능한 것은 아닙니다. 지정 운임 구역·편도/왕복 횟수 등 조건 연결이 확보되지 않은 패스는 자동 비교를 끈 상태로 보관합니다.

저장된 planId와 결과가 작업의 SUCCEEDED 상태로 전달됩니다. 완료 팝업·닫기/일정 보기·표시 시간은 프론트의 작업 상태 조회와 화면 로직이 담당합니다. 생성 중 홈으로 나가거나 화면을 닫는 동작과 생성 취소 요청은 별개입니다.

3. 재조회·재생성·취소

상황	처리
저장 일정 재조회	회원 소유권 확인 → 방문 항목/교통 메타데이터 복원 → 상세·한국어 설명·운임·경로 보강 → 비용/패스 재계산
다시 생성	해당 회원의 일정인지 확인하고 새 요청 조건으로 생성·저장 처리
취소 요청	해당 회원 작업만 중단 요청. 실행 중에는 취소 확인 지점에서 중단하고 저장 트랜잭션도 취소 여부 확인
저장 마무리 중 취소	409 반환. 저장 완료 후 일반 삭제로 처리 가능
서버 재시작	미완료 메모리 작업은 복구 실행하지 않음. 저장된 일정과 완료 요청은 DB에서 조회 가능

4. 예전 코드와 실제 사용하는 코드 구분

PlanService에는 selectCandidates, calculateTspWithTimeWindows, 지리 클러스터링 및 경로 반전 함수가 남아 있습니다. 현재 PlanController.createPlan은 이 TSP 생성 경로를 호출하지 않고 도시별 후보를 runScheduleSimulation에 직접 전달합니다. 클래스 주석의 '13단계', 'TSP-TW' 표현만 보고 현재 엔진을 K-means + 2-opt 파이프라인으로 설명하면 실제 호출과 다릅니다.

현재 도시 순서 최적화는 도시 대표점 사이 거리 기준으로 정확히 계산하지만, 하루 방문 순서는 가능한 후보를 순차 선택하는 근사 방식입니다. 전체 여행의 실제 교통 최단시간·최저비용을 보장하는 전역 최적화는 아닙니다.

5. 주요 소스 파일 찾아보기

아래 링크는 현재 PC의 코드이며 GitHub 원본과 다를 수 있습니다.

- PlanGenerationController: public PlanGenerationJobs.Result submit (controller/PlanGenerationController.java:23)
- PlanGenerationJobs: public synchronized Result submit (service/PlanGenerationJobs.java:40)
- PlanController: public AiRouteResponse createPlan (controller/PlanController.java:74)
- PlanService: public DayWindow dayWindow (service/PlanService.java:48)
- PlanService: public List<Place> applyWeightedScoring (service/PlanService.java:119)
- CityDayPlanner: public static List<String> citiesForDays (util/CityDayPlanner.java:76)
- StayPlanner: public Stay resolve (service/StayPlanner.java:19)
- TransportService: public TransportLegDto between (service/TransportService.java:19)
- PlanService: public SimulationResult runScheduleSimulation (service/PlanService.java:324)
- OpeningHoursParser: public static (util/OpeningHoursParser.java:14)
- DayBoundaryService: public void append (service/DayBoundaryService.java:19)
- WeatherService: public int score (service/WeatherService.java:72)
- DistanceTransportEstimateService: public static TransportLegDto estimate (service/DistanceTransportEstimateService.java:32)
- RouteGeometryService: public void enrich (service/RouteGeometryService.java:10)
- TransitCostService: public Summary summarize (service/TransitCostService.java:107)
- SavedPlanController: public (controller/SavedPlanController.java:18)

6. 마지막 GitHub 확인 결과

2026-10-09에 두 저장소의 원격 브랜치 HEAD를 직접 확인하고 검토용 체크아웃을 fetch했습니다. 사용자 실행 폴더는 Git 저장소가 아닌 로컬 수정 복사본이므로 'git pull 완료' 또는 '원격과 동일'로 표현할 수 없습니다.

저장소/브랜치	현재 원격 커밋	커밋 시각(KST)	이전 검토 대비
A5front / main	dc3d1649b20adb15863679f53bf03ce2de206f6c	2026-10-05 14:36:40	새 커밋 없음
TravelPlanner_Backend / refactor	bb9abc6e560dcf9ea0bd8d7aaaccd7ee5c06fedd	2026-10-09 16:38:52	1211e9c 이후 15개 커밋, 50개 파일 변경
TravelPlanner_Backend / master	26ee93d4f644b9b0ac5b5172294df68458987bfc	2026-09-13 21:57:29	refactor보다 오래된 별도 기본 브랜치

프론트: https://github.com/gunyi0624/A5front/commit/dc3d1649b20adb15863679f53bf03ce2de206f6c

백엔드: https://github.com/NaDongHwi/TravelPlanner_Backend/commit/bb9abc6e560dcf9ea0bd8d7aaaccd7ee5c06fedd

변경 설명: https://github.com/NaDongHwi/TravelPlanner_Backend/blob/bb9abc6e560dcf9ea0bd8d7aaaccd7ee5c06fedd/CHANGES.md

검토 범위는 원격 커밋·변경 파일 목록·CHANGES.md 및 핵심 생성 서비스/엔진 코드입니다. 원격 50개 파일 전체의 통합 테스트나 보안 감사를 수행한 것은 아닙니다.

원격에서 새로 확인한 주요 내용

• 컨트롤러에서 생성·저장·타임라인 조립을 PlanGenerationService, PlanPersistenceService, TimelineAssembler로 분리.

• 구조화 영업시간, 테마 어휘 추정과 테마 균형, 고정 일정 날짜/위치, 저장 조건 복원 보강.

• 도시별 관광 분량을 고려한 일수 배분, 늦은 야외 방문·긴 식당 이동 억제.

• 인증 실패 상태 코드, 관리자 권한, PDF/재탐색 소유권 등 보강 변경.

이번 반영 판단

이번 요청은 마지막 확인과 현재 알고리즘 문서 작성이므로 원격 코드를 실행 폴더에 통째로 교체하지 않았습니다. 두 프로젝트의 실행 코드는 이번 확인으로 변경하지 않았습니다.

원격은 새 엔티티 컬럼과 서비스 구조를 요구하고, 로컬의 비동기 생성·취소·교통 DB·패스 비교와 연결되지 않은 다른 생성 경로입니다. 특히 원격의 식사 간격 90분, 입국 준비 90분/출국 준비 120분, 숙소 복귀 45분 초과 허용은 현재 사용자 설정에 맞춘 식사 300분(5끼 150분), 입국 120분/출국 180분, 숙소 도착 목표와 다릅니다. 원격의 4·5회 식사에는 간식도 포함돼 현재 식사/간식 분리와도 다릅니다.

다음 통합 때는 구조화 영업시간·테마 분류·관광 분량 기반 일수 배분을 개별 검토하고, 기존 모바일 연동·식사 간격·숙소 마감·취소/저장 경계를 유지하는 회귀 검증을 함께 해야 합니다. 위 원격 개선을 현재 이미 적용된 기능으로 읽으면 안 됩니다.